

1. 递归的两个必要条件（缺一不可）

写递归就像下楼梯，你必须知道什么时候停下来，否则就会掉进无限循环，导致**栈溢出（Stack Overflow）**。

1. **基准情形 (Base Case)**：也就是“停止条件”。必须有一个情况是不再递归，直接返回结果。
 - 在二叉树中：通常是 `if (root == NULL)`。
 2. **递归步骤 (Recursive Step)**：把大问题分解成规模更小、但形式相同的子问题，且这些子问题必须正在向“基准情形”靠拢。
 - 在二叉树中：处理完根节点，去处理 `root->left` 和 `root->right`。
-

2. 递归的执行过程：函数调用栈

理解递归的关键在于理解“**递**”去和“**归**”来。

- **递 (Forward)**：每次调用函数，系统都会在内存中开辟一块空间（栈帧），保存当前的变量、返回地址等。
- **归 (Backward)**：当遇到基准情形时，函数开始层层返回，释放栈空间，并将结果带回给上一层调用者。

3. 写好递归的小技巧

1. **不要试图在脑子里人肉模拟超过 3 层的递归！** 这样会晕。
 2. **信任你的函数**：在写 `root->left = buildTree(...)` 时，你要假设这个函数已经写好了，它一定能返回正确的左子树给你，你只需要关注当前这一层怎么组合即可。
 3. **先写 Base Case**：动笔写递归函数的第一行，永远应该是判断什么时候结束。
-

4. 掌握递归思维：“雇佣兵思维”

4.1. 思维核心：信任与分工

假设你是一个包工头（当前函数），你的任务是给一棵大树数有多少个果子。

- **你的原则**：我很懒，我只数根节点上那一个果子。
- **你的手段**：你有两个雇佣兵（左分身和右分身）。
- **递归逻辑**：
 -

1. 你问左分身：“喂，你负责的那部分（左子树）有多少果子？”
-
2. 你问右分身：“喂，你负责的那部分（右子树）有多少果子？”
-
3. **信任：** 你不需要管他们是怎么数的，你坚信他们一定会给你一个准确的数字。
-
4. **汇总：** 最终结果 = 左分身的数 + 右分身的数 + 我手里的这 1 个。
-
5. **出口：** 如果分身汇报说“头儿，我面前根本没有树（NULL）”，那他直接告诉你 0 即可。

什么时候用？ 只要你发现一个大问题的子问题，长得跟大问题一模一样，就可以雇佣分身。

4.2.三个经典场景：帮你找到“调用时机”

场景 A：当你需要“拆地雷” (类似本题建树)

如果你发现要解决当前问题，必须先知道“左边”和“右边”的结果。

- **例子：** 判断两棵树是否完全相同。
- **思路：** * 我先看我这层的根节点值一样吗？
 - 如果一样，我得问分身：“左子树一样吗？”
 - 再问：“右子树一样吗？”
 - **代码感觉：** `return (我这层OK) && (递归左边) && (递归右边);`

场景 B：当你遇到“套娃”结构 (路径问题)

如果你发现每走一步，剩下的路虽然变短了，但规则没变。

- **例子：** 走迷宫。
- **思路：** * 我站在这里，往上下左右看一眼。
 - 只要有一个方向能走通，我就派个分身去那一格继续走。
 - **代码感觉：** `if (can_go) return solve(下一格);`

场景 C：当你需要“事后处理” (类似本题后序遍历)

如果你必须等手下把事情全办完了，你才能做决定。

- **例子：** 删除一棵树。
- **思路：** * 我不能先删根节点，删了根我就找不到孩子了。
 - 我先让左分身把左边删光，让右分身把右边删光。
 - 最后，我再把自己（根）删掉。

5. 经典例题

5.1. 数学逻辑型：阶乘 (Factorial)

这是最简单的递归，用来理解“传递”与“回溯”。

- **问题：** 计算 $n! = n \times (n-1) \times \cdots \times 1$ 。
- **关系式：** $n! = n \times (n-1)!$
- **分身逻辑：** * 我（计算 n 的阶乘）只管把 n 乘上“分身”算出来的结果。
 - 分身（计算 $n-1$ 的阶乘）再去管他的分身。
- **出口：** 当 $n = 1$ 时，别再找分身了，直接告诉上级结果是 1。

代码实现

```
int factorial(int n) {  
    if (n == 1) return 1;           // 出口：最简单的情况  
    return n * factorial(n - 1);    // 关系式：我 × (n-1)的结果  
}
```

5.2. 数组搜索型：二分查找 (Binary Search)

这是理解“缩小规模”的典型例子。

- **问题：** 在有序数组中找数字 `target`。
- **分身逻辑：**
 - 我先看数组中间那个数。
 - 如果中间的数比 `target` 大，说明 `target` 在左半边。
 - 我雇佣一个分身，**只让他看左半边**。规则完全不变。
- **出口：** * 找到了：返回下标。
 - 找完了（范围空了）：返回 -1。

代码实现

```
int binarySearch(int arr[], int L, int R, int target) {  
    if (L > R) return -1; // 出口：找不到了  
  
    int mid = L + (R - L) / 2;
```

```

    if (arr[mid] == target) return mid; // 出口：找到了

    if (arr[mid] > target)
        return binarySearch(arr, L, mid - 1, target); // 递归左边
    else
        return binarySearch(arr, mid + 1, R, target); // 递归右边
}

```

5.3.翻转二叉树（把所有节点的左右子树对调）

逻辑应该是：

1. **信任：** 我有一个函数 `invert(node)`，它能把一棵树变镜像。
2. **动作：** * 我先把我的左孩子和右孩子交换位置（拿个临时变量 `temp` 交换）。
 - 交换完后，我发现左孩子和右孩子**内部**还没翻转。
 - **调用递归：** `invert(root->left);` 和 `invert(root->right);`（交给分身去搞定它们内部的翻转）。
3. **出口：** 如果节点是空的，直接返回。

代码实现（伪代码）:

```

void invert(Node* root) {
    if (root == NULL) return; // 出口

    // 交换左右孩子
    Node* temp = root->left;
    root->left = root->right;
    root->right = temp;

    // 信任递归：剩下的交给分身
    invert(root->left);
    invert(root->right);
}

```

tips:

翻转和对调的顺序并不重要，两者都可以实现相同的效果。

根据你把“自己要做的事”放在“递归调用”的什么位置，会有不同的效果：

1. **事发前处理（先序）：** 先处理根，再处理子树（如：从上往下传值、翻转）。

2. **事中处理（中序）**：处理完左边，处理根，再处理右边（常用于二叉搜索树，因为结果是有顺序的）。
3. **事后处理（后序）**：先处理完两个子树，最后处理根（如：计算树的高度、删除整棵树）。

6. 递归的精髓：不要越级指挥

很多初学者容易犯的错误是：试图在这一层就把下一层的事儿给干了。

- **错误思维**：我要交换 `root->left` 和 `root->right`，然后我还要交换 `root->left->left` 和 `root->left->right`
- **正确递归思维**：我只负责交换 `root` 的两个孩子。至于孩子们自己的孩子怎么换，那是下一层调用该操心的事。

“我只管我能看到的这一层，看不见的交给递归。” —— 这就是掌握递归的终极秘诀。